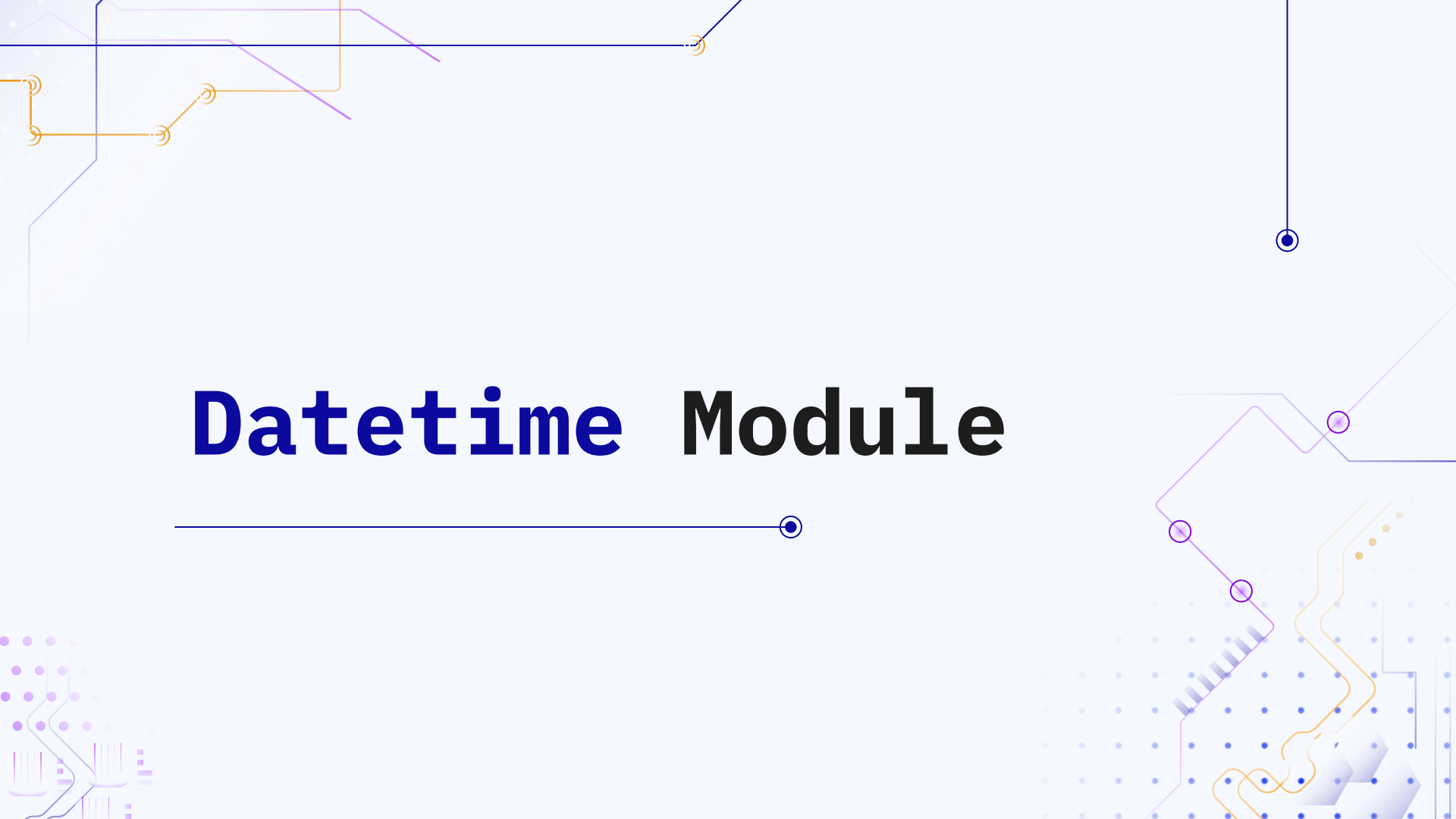


The background features a light blue gradient with abstract circuit-like patterns. These patterns consist of thin lines in blue, orange, and purple, some of which are connected to small circles. There are also clusters of small dots in these colors, particularly in the bottom left and bottom right corners. The overall aesthetic is technical and modern.

Datetime Module

Introduction

Python's built-in **datetime** module provides classes to work with:

- Dates (year-month-day)
- Times (hour-minute-second-microsecond)
- Date + time together
- Durations / differences between two dates/times
- Timezone-aware date-times (when needed)

Officially, it's designed for easy date/time manipulation + formatting output.

Syntax

```
import datetime
# OR (recommended for beginners)
from datetime import date, time, datetime, timedelta, timezone
```

A) dates (only calendar date)

Stores year, month, day

Syntax

```
from datetime import date

d = date(2026, 2, 10)
print(d)           # 2026-02-10
print(d.year)      # 2026
print(d.month)     # 2
print(d.day)       # 10
print(date.today())
```

B) time (only time of day)

Stores time independent of the date

Syntax

```
from datetime import time  
  
t = time(14, 30, 15)  
print(t)           # 14:30:15  
print(t.hour, t.minute, t.second) # 14 30 15
```

C) datetime (date + time together)

This is the most-used class in real programs.

Syntax

```
from datetime import datetime

dt = datetime(2026, 2, 10, 14, 30, 15)
print(dt)           # 2026-02-10 14:30:15
print(dt.date())    # 2026-02-10
print(dt.time())    # 14:30:15
print(datetime.now()) # 2026-02-12 13:03:30.818740
```

D) timedelta (duration / difference)

timedelta represents a time gap (like “3 days”, “2 hours”, etc.).

Syntax

```
from datetime import datetime, timedelta

now = datetime.now()
print(now + timedelta(days=7))      # 2026-02-19 13:06:02.520177
print(now - timedelta(hours=2))     # 2026-02-12 11:06:02.520177
```

Formatting dates/times

- **strftime()** converts a date/datetime into a nicely formatted string.
- **It takes a format string to control output.**

Syntax

```
from datetime import datetime, timedelta

dt = datetime.now()
print(dt.strftime("%Y-%m-%d"))      # 2026-02-12
print(dt.strftime("%d/%m/%Y"))      # 12/02/2026
print(dt.strftime("%I:%M %p"))      # 01:08 PM
print(dt.strftime("%A, %B %d, %Y")) # Thursday, February 12, 2026
```

Formatting dates/times

Most useful format codes

- **%Y** year (4-digit)
- **%m** month (01–12)
- **%d** day (01–31)
- **%H** hour 24-hr (00–23)
- **%I** hour 12-hr (01–12)
- **%M** minutes,
- **%S** seconds
- **%p** AM/PM
- **%A** weekday name,
- **%B** month name

Parsing (string → datetime)

strptime() converts a date/time string into a real datetime object (so you can compare, subtract, add time, etc.).

Syntax

```
from datetime import datetime, timedelta  
  
s = "10-02-2026 14:30:15"  
dt = datetime.strptime(s, "%d-%m-%Y %H:%M:%S")  
print(dt) # 2026-02-10 14:30:15
```

Your format must match the string exactly

Common Operations

A) combine() (date + time → datetime)

Syntax

```
from datetime import date, time, datetime  
  
dt = datetime.combine(date(2026,2,10), time(9,15))  
print(dt) # 2026-02-10 09:15:00
```

Common Operations

B) `replace()` (change parts without rebuilding everything)

Syntax

```
from datetime import datetime  
  
dt = datetime(2026, 2, 10, 9, 15)  
print(dt.replace(hour=10, minute=0)) # 2026-02-10 10:00:00
```

Common Operations

c) Comparisons & sorting

Syntax

```
from datetime import date  
  
print(date(2026,2,10) > date(2026,2,1)) # True
```



Practice Set – 1

Download Link in **Description** and **Pinned Comment**

WATCH

Level up your coding with each episode in this focused Python series.



Next Video!

**Practice Set - 1
Solution**

